

DevOps and Automations TASK – 1

- 1. Install Jenkins on your local machine or use a Jenkins Docker container, then access the Jenkins dashboard and take a screenshot of the home page to confirm your setup.**

Ans :

1. Install Docker Desktop

Download and install Docker Desktop from Docker Desktop official site.

Open Docker Desktop and make sure it is running.

2. Open Terminal / PowerShell

On Windows, open PowerShell.
On macOS/Linux, open Terminal.

3. Download and start Jenkins

Run:

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 -v  
jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

4. Check Jenkins is running

Run:

```
docker ps
```

5. Get the Jenkins password

Run:

```
docker exec jenkins cat  
/var/jenkins_home/secrets/initialAdminPassword
```

6. Open Jenkins

Open your browser and enter:

<http://localhost:8080>

7. Unlock Jenkins

You will see Unlock Jenkins.

Paste the password you copied in Step 5 and click Continue.

8. Install plugins

Select:

Install suggested plugins

Wait for the installation to finish.

9. Create your Jenkins administrator account

Enter:

- Username: Choose a username, e.g. admin
- Password: Create a password
- Full name: Your name
- Email: Your email address

Click Save and Continue.

10. Finish Jenkins setup

Click Save and Finish

2. Create a new freestyle Jenkins job named 'SpotifyPlaylistBuild' that simply echoes 'Building Spotify Playlist App' in the build step, then run the job and check the console output.

Ans :

- Open your browser.
- Go to:

- `http://localhost:8080`
- Log in to Jenkins.
- Click New Item.
- Enter:
- SpotifyPlaylistBuild
- Select Freestyle project.
- Click OK.
- Scroll down to Build Steps.
- Click Add build step.
- Select Execute Windows batch command if you are using Windows.
- Enter:
- `echo Building Spotify Playlist App`
- Click Save.
- Click Build Now.
- Wait for the build to finish.
- Under Build History, click #1.
- Click Console Output.

**3. Set up a basic Jenkins pipeline (using Pipeline script) that checks out code from any public GitHub repository (for example, a simple JavaScript or Python app), then runs a build step using 'echo' to simulate a build process.

Hint: Use the 'pipeline' syntax with 'stages' for checkout and build.**

Ans :

- Open Jenkins:
- `http://localhost:8080`
- Log in to Jenkins.
- Click New Item.
- Enter the name:
- `GitHubCheckoutPipeline`
- Select:
- Pipeline
- Click OK.
- Scroll down to Pipeline.
- In Definition, select:
- Pipeline script
- In the Script box, delete the existing content.
- Copy and paste this complete pipeline:
- `pipeline {`
- `agent any`
- `stages {`
 - `stage('Checkout') {`
 - `steps {`
 - `git 'https://github.com/octocat/Hello-World.git'`
 - `}`
 - `}`
- `stage('Build') {`

- steps {
 - echo 'Building application...'
 - }
 - }
- }
- }
- Click Save.
- Click Build Now.
- Wait for the pipeline to finish.
- Under Build History, click the build number, for example:
- #1
- Click Console Output.
- Check that the output shows that the GitHub repository was checked out successfully.
- Check that the output contains:
- Building application...
- At the end, confirm:
- Finished: SUCCESS

**4. Modify your Jenkins pipeline to add a test stage that runs a simple test command (for example, 'npm test' for a Node.js app or 'python -m unittest' for Python), and configure the pipeline to fail if tests do not pass.

Hint: If you don't have real tests, use a dummy command like 'exit 1' to simulate a failure and see how Jenkins handles it.**

Ans :

- Open Jenkins:

- <http://localhost:8080>
- Log in to Jenkins.
- Open the pipeline you created in Task 3:
- GitHubCheckoutPipeline
- Click Configure.
- Scroll down to Pipeline.
- Make sure Definition is set to:
- Pipeline script
- Replace the existing pipeline script with this:
- pipeline {
- agent any
- stages {
 - stage('Checkout') {
 - steps {
 - git 'https://github.com/octocat/Hello-World.git'
 - }
 - }
 - stage('Build') {
 - steps {
 - echo 'Building application...'
 - }
 - }
 - stage('Test') {

- steps {
 - echo 'Running tests...'
 - sh 'exit 0'
 - }
 - }
- }
- }
- The Test stage is added after the Build stage.
- sh 'exit 0' simulates a successful test.
- Click Save.
- Click Build Now.
- Wait for the pipeline to finish.
- Click the new build number under Build History, for example:
- #2
- Click Console Output.
- Check that you can see:
- Building application...
- Running tests...
- Finished: SUCCESS
- Test the failure condition
- Go back to Configure.
- Replace:
- sh 'exit 0'
- with:
- sh 'exit 1'
- Click Save.

- Click Build Now again.
- Open the new build from Build History.
- Click Console Output.
- You should see something similar to:
 - Running tests...
 - script returned exit code 1
 - Finished: FAILURE
- This demonstrates that Jenkins fails the pipeline when the test command fails.

5. Use ChatGPT or GitHub Copilot to help you write a Jenkinsfile that builds and tests a Zomato-style food ordering app (can be a simple placeholder app), then copy and use this Jenkinsfile in a new pipeline job and describe any errors or improvements suggested by the AI tool.

Ans :

- Create a new Jenkins pipeline
- Open:
 - <http://localhost:8080>
- Then:
 - Click New Item.
 - Enter:
 - ZomatoFoodOrderingPipeline
 - Select Pipeline.
 - Click OK.
 - Scroll down to Pipeline.
 - Set Definition to:

- Pipeline script
- Copy this Jenkinsfile into the Pipeline Script box
- pipeline {
- agent any
- stages {
 - stage('Checkout') {
 - steps {
 - git 'https://github.com/octocat/Hello-World.git'
 - }
 - }
 - stage('Install Dependencies') {
 - steps {
 - echo 'Installing dependencies for food ordering app...'
 - }
 - }
 - stage('Build') {
 - steps {
 - echo 'Building Zomato-style food ordering app...'
 - }
 - }
 - stage('Test') {
 - steps {
 - echo 'Testing food ordering features...'
 - sh 'echo "Restaurant search test passed"'
 - sh 'echo "Food ordering test passed"'

- sh 'echo "Cart test passed"'
 - }
 - }
- }
- post {
 - success {
 - echo 'Zomato-style food ordering app build and tests completed successfully!'
 - }
 - failure {
 - echo 'Build or tests failed. Check the console output.'
 - }
- }
- }
- Save the pipeline
- Click:
- Save
- Run the pipeline
- Click:
- Build Now
- Wait for Jenkins to finish.
- Check the result
- Under Build History, click the build number, such as:
- #1
- Then click:
- Console Output
- You should see messages similar to:
- Installing dependencies for food ordering app...
- Building Zomato-style food ordering app...
- Testing food ordering features...

- Restaurant search test passed
- Food ordering test passed
- Cart test passed
- Zomato-style food ordering app build and tests completed successfully!
- Finished: SUCCESS